4.3 Recursion Run Amok

Although recursion is a very powerful tool, it can easily be misused in various ways. In this section, we examine several problems in which a poorly implemented recursion causes drastic inefficiency, and we discuss some strategies for recognizing and avoid such pitfalls.

We begin by revisiting the ***element uniqueness problem***, defined on page 135 of Section 3.3.3. We can use the following recursive formulation to determine if all n elements of a sequence are unique. As a base case, when $n = 1$, the elements are trivially unique. For $n \geq 2$, the elements are unique if and only if the first $n-1$ elements are unique, the last $n-1$ items are unique, and the first and last elements are different (as that is the only pair that was not already checked as a subcase). A recursive implementation based on this idea is given in Code Fragment 4.6, named unique3 (to differentiate it from unique1 and unique2 from Chapter 3).

```
1  def unique3(S, start, stop):
2    """Return True if there are no duplicate elements in slice S[start:stop]."""
3    if stop - start <= 1: return True                 # at most one item
4    elif not unique(S, start, stop-1): return False   # first part has duplicate
5    elif not unique(S, start+1, stop): return False   # second part has duplicate
6    else: return S[start] != S[stop-1]                # do first and last differ?
```

Code Fragment 4.6: Recursive unique3 for testing element uniqueness.

Unfortunately, this is a terribly inefficient use of recursion. The nonrecursive part of each call uses $O(1)$ time, so the overall running time will be proportional to the total number of recursive invocations. To analyze the problem, we let n denote the number of entries under consideration, that is, let n= stop − start.

If $n = 1$, then the running time of unique3 is $O(1)$, since there are no recursive calls for this case. In the general case, the important observation is that a single call to unique3 for a problem of size n may result in two recursive calls on problems of size $n-1$. Those two calls with size $n-1$ could in turn result in four calls (two each) with a range of size $n-2$, and thus eight calls with size $n-3$ and so on. Thus, in the worst case, the total number of function calls is given by the geometric summation

$$1 + 2 + 4 + \cdots + 2^{n-1},$$

which is equal to $2^n - 1$ by Proposition 3.5. Thus, the running time of function unique3 is $O(2^n)$. This is an incredibly inefficient function for solving the element uniqueness problem. Its inefficiency comes not from the fact that it uses recursion—it comes from the fact that it uses recursion poorly, which is something we address in Exercise C-4.11.

An Inefficient Recursion for Computing Fibonacci Numbers

In Section 1.8, we introduced a process for generating the Fibonacci numbers, which can be defined recursively as follows:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-2} + F_{n-1} \quad \text{for } n > 1. \end{aligned}$$

Ironically, a direct implementation based on this definition results in the function bad_fibonacci shown in Code Fragment 4.7, which computes the sequence of Fibonacci numbers by making two recursive calls in each non-base case.

```
1  def bad_fibonacci(n):
2    """Return the nth Fibonacci number."""
3    if n <= 1:
4      return n
5    else:
6      return bad_fibonacci(n-2) + bad_fibonacci(n-1)
```

Code Fragment 4.7: Computing the nth Fibonacci number using binary recursion.

Unfortunately, such a direct implementation of the Fibonacci formula results in a terribly inefficient function. Computing the nth Fibonacci number in this way requires an exponential number of calls to the function. Specifically, let c_n denote the number of calls performed in the execution of bad_fibonacci(n). Then, we have the following values for the c_n's:

$$\begin{aligned} c_0 &= 1 \\ c_1 &= 1 \\ c_2 &= 1 + c_0 + c_1 = 1 + 1 + 1 = 3 \\ c_3 &= 1 + c_1 + c_2 = 1 + 1 + 3 = 5 \\ c_4 &= 1 + c_2 + c_3 = 1 + 3 + 5 = 9 \\ c_5 &= 1 + c_3 + c_4 = 1 + 5 + 9 = 15 \\ c_6 &= 1 + c_4 + c_5 = 1 + 9 + 15 = 25 \\ c_7 &= 1 + c_5 + c_6 = 1 + 15 + 25 = 41 \\ c_8 &= 1 + c_6 + c_7 = 1 + 25 + 41 = 67 \end{aligned}$$

If we follow the pattern forward, we see that the number of calls more than doubles for each two consecutive indices. That is, c_4 is more than twice c_2, c_5 is more than twice c_3, c_6 is more than twice c_4, and so on. Thus, $c_n > 2^{n/2}$, which means that bad_fibonacci(n) makes a number of calls that is exponential in n.

An Efficient Recursion for Computing Fibonacci Numbers

We were tempted into using the bad recursion formulation because of the way the nth Fibonacci number, F_n, depends on the two previous values, F_{n-2} and F_{n-1}. But notice that after computing F_{n-2}, the call to compute F_{n-1} requires its own recursive call to compute F_{n-2}, as it does not have knowledge of the value of F_{n-2} that was computed at the earlier level of recursion. That is duplicative work. Worse yet, both of those calls will need to (re)compute the value of F_{n-3}, as will the computation of F_{n-1}. This snowballing effect is what leads to the exponential running time of bad_recursion.

We can compute F_n much more efficiently using a recursion in which each invocation makes only one recursive call. To do so, we need to redefine the expectations of the function. Rather than having the function return a single value, which is the nth Fibonacci number, we define a recursive function that returns a pair of consecutive Fibonacci numbers (F_n, F_{n-1}), using the convention $F_{-1} = 0$. Although it seems to be a greater burden to report two consecutive Fibonacci numbers instead of one, passing this extra information from one level of the recursion to the next makes it much easier to continue the process. (It allows us to avoid having to recompute the second value that was already known within the recursion.) An implementation based on this strategy is given in Code Fragment 4.8.

```
1  def good_fibonacci(n):
2    """Return pair of Fibonacci numbers, F(n) and F(n-1)."""
3    if n <= 1:
4      return (n,0)
5    else:
6      (a, b) = good_fibonacci(n-1)
7      return (a+b, a)
```

Code Fragment 4.8: Computing the nth Fibonacci number using linear recursion.

In terms of efficiency, the difference between the bad recursion and the good recursion for this problem is like night and day. The bad_fibonacci function uses exponential time. We claim that the execution of function good_fibonacci(n) takes $O(n)$ time. Each recursive call to good_fibonacci decreases the argument n by 1; therefore, a recursion trace includes a series of n function calls. Because the nonrecursive work for each call uses constant time, the overall computation executes in $O(n)$ time.

4.3.1 Maximum Recursive Depth in Python

Another danger in the misuse of recursion is known as ***infinite recursion***. If each recursive call makes another recursive call, without ever reaching a base case, then we have an infinite series of such calls. This is a fatal error. An infinite recursion can quickly swamp computing resources, not only due to rapid use of the CPU, but because each successive call creates an activation record requiring additional memory. A blatant example of an ill-formed recursion is the following:

```
def fib(n):
  return fib(n)                    # fib(n) equals fib(n)
```

However, there are far more subtle errors that can lead to an infinite recursion. Revisiting our implementation of binary search in Code Fragment 4.3, in the final case (line 17) we make a recursive call on the right portion of the sequence, in particular going from index mid+1 to high. Had that line instead been written as

```
    return binary_search(data, target, mid, high)      # note the use of mid
```

this could result in an infinite recursion. In particular, when searching a range of two elements, it becomes possible to make a recursive call on the identical range.

A programmer should ensure that each recursive call is in some way progressing toward a base case (for example, by having a parameter value that decreases with each call). However, to combat against infinite recursions, the designers of Python made an intentional decision to limit the overall number of function activations that can be simultaneously active. The precise value of this limit depends upon the Python distribution, but a typical default value is 1000. If this limit is reached, the Python interpreter raises a RuntimeError with a message, `maximum recursion depth exceeded`.

For many legitimate applications of recursion, a limit of 1000 nested function calls suffices. For example, our binary_search function (Section 4.1.3) has $O(\log n)$ recursive depth, and so for the default recursive limit to be reached, there would need to be 2^{1000} elements (far, far more than the estimated number of atoms in the universe). However, in the next section we discuss several algorithms that have recursive depth proportional to n. Python's artificial limit on the recursive depth could disrupt such otherwise legitimate computations.

Fortunately, the Python interpreter can be dynamically reconfigured to change the default recursive limit. This is done through use of a module named sys, which supports a getrecursionlimit function and a setrecursionlimit. Sample usage of those functions is demonstrated as follows:

```
import sys
old = sys.getrecursionlimit( )       # perhaps 1000 is typical
sys.setrecursionlimit(1000000)       # change to allow 1 million nested calls
```